Section A: LLM Foundations & Hugging Face

1. **Hugging Face Setup & Text Generation**

```
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
[{'generated_text': 'AI is transforming industries by providing them with all the
resources to help create them.\u202e So as the number of jobs rises, more businesses
become engaged. As the number of workers increase, more people'}]
```

```
[{'generated_text': 'AI is transforming industries by putting more and more of its
workers onto this kind of industry.\n\n\n\n\n\n\n\nThere are more than 200 new jobs in
this area, although one million'}]
```

```
[{'generated_text': "AI is transforming industries by adding $80 billion to the state
government's debt, which means those who do not have enough credit are going to lose that
money.\n\n\nThe government is now asking"}]
```

Observation:

Even for same input prompt, output can vary.

2. **Tokenization Demo**

```
tokenizer_config.json:   0%|          | 0.00/48.0 [00:00<?, ?B/s]
config.json:   0%|          | 0.00/570 [00:00<?, ?B/s]
vocab.txt: 0.00B [00:00, ?B/s]
tokenizer.json: 0.00B [00:00, ?B/s]
Original Sentence:
LLMs are powerful tools for natural language understanding.

Tokens:
['ll', '##ms', 'are', 'powerful', 'tools', 'for', 'natural', 'language', 'understanding',
'.']

Token IDs:
[2222, 5244, 2024, 3928, 5906, 2005, 3019, 2653, 4824, 1012]

Sequence Length (including special tokens):
12
```

Observation:

Token count need not be equal to number of words in sentence. Tokenizer frameworks have their own logic to split sentences and words into token for maximum efficiency.

Section B: Prompt Engineering

3. **Prompt Tuning Challenge**

First Prompt:

```
Prompt: summarize: Artificial Intelligence is transforming industries by automating
tasks, improving decision-making, and enabling new innovations across sectors.
Output: Artificial Intelligence (AI) is transforming industries by automating tasks,
improving decision-making, and enabling new innovations across sectors.

Prompt: question: What is the capital of France?
Output: paris

Prompt: Write a poem about AI.
Output: it 's not always easy to find a place for me in the world i know and love i'm
always afraid of it i know that there are no answers i know that i have no one to talk to
i know that no one will believe me
```

After Tuning:

Output

```
Prompt: extract keywords: Artificial Intelligence is transforming industries by
automating tasks, improving decision-making, and enabling new innovations across sectors.
Output: Artificial Intelligence industry

Prompt: question: What is the capital of France? answer in 3 words.
Output: paris

Prompt: Write a 4-line poem about AI where each line starts with AI.
Output: i can n't think of a better word to describe the little girl than aili
```

Observation:

- Rephrasing prompts improved output quality and consistency. More specific instructions led to better adherence to constraints such as word limits and formatting.
- For summarisation, explicitly mentioning the word limit and focus helped reduce unnecessary details.
- In Q&A tasks, instructing the model to provide brief, factual responses resulted in more precise answers compared to generic prompts.
- For creative generation, adding structural guidance ensured the output followed the desired format, whereas vague prompts produced inconsistent results.
- Overall, prompt clarity, constraints, and formatting instructions play a crucial role in guiding LLM behaviour.

- However, lot depends on choosing appropriate model as well. Even small changes in wording can shift outputs from generic and verbose to precise and task-aligned in large models but when it comes to small models; it seems sometimes these models are ignoring slight variation in prompt.

Section C: Embeddings with Gensim

4. **Word Embeddings**

```
Word: king
First 10 vector values:
[ 0.50451   0.68607  -0.59517  -0.022801  0.60046  -0.13498  -0.08813
  0.47377  -0.61798  -0.31012 ]

Top 5 similar words:
prince: 0.8236
queen: 0.7839
ii: 0.7746
emperor: 0.7736
son: 0.7667

Word: queen
First 10 vector values:
[ 0.37854   1.8233   -1.2648   -0.1043    0.35829   0.60029  -0.17538
  0.83767  -0.056798 -0.75795 ]
```

```
Top 5 similar words:
princess: 0.8515
lady: 0.8051
elizabeth: 0.7873
king: 0.7839
prince: 0.7822

Word: diamond
First 10 vector values:
[-0.4958   0.78421 -0.606    1.3967   0.28888 -0.2058  -0.10745 -0.33252
  1.3608   0.15091]

Top 5 similar words:
gold: 0.7715
diamonds: 0.7663
gem: 0.7375
silver: 0.7210
jewel: 0.7102
```

Observation:

Based on vector embeddings, models are able to generate similar word efficiently with high matching score.

5. **Sentence-Level Embeddings**

Output

```
Cosine Similarity Matrix:

[[1.0000001  0.85567063 0.7741947  0.72519153 0.7883498 ]
 [0.85567063 0.9999998  0.79679334 0.6424331  0.738606  ]
 [0.7741947  0.79679334 1.0000001  0.5328553  0.6170783 ]
 [0.72519153 0.6424331  0.5328553  0.9999999  0.9284023 ]
 [0.7883498  0.738606   0.6170783  0.9284023  1.0000001 ]]
```

Observation:

Similarity matrix values for same domain sentences (AI related or jewellery related) are higher than cross-domain sentences (AI vs Jewellery).

Section D: Application Exploration

6. **Transformer Application**

```
Text: The product quality is excellent and delivery was super fast!
Sentiment: POSITIVE
--------------------------------------------------
Text: I am very disappointed with the customer service.
Sentiment: NEGATIVE
--------------------------------------------------
Text: The experience was okay, nothing special but not bad either.
Sentiment: POSITIVE
--------------------------------------------------
Text: Absolutely loved the user interface and ease of use!
Sentiment: POSITIVE
--------------------------------------------------
Text: The app keeps crashing and it's very frustrating.
Sentiment: NEGATIVE
--------------------------------------------------
```

Observation:

- Sentiment classification using Transformers can significantly improve how businesses handle customer feedback at scale.
- Instead of manually reviewing thousands of reviews, companies can automatically categorize feedback into positive, negative, or neutral sentiments in real time.
- This enables faster response to dissatisfied customers, helping improve retention and brand reputation.
- It can also identify common pain points, such as product issues or service delays, by analyzing trends in negative sentiment.

- Additionally, marketing teams can use positive feedback to highlight strengths in campaigns. Overall, this application enhances decision-making, operational efficiency, and customer experience across professional workflows.

Key observations about LLMs vs. embeddings:

Embeddings help machines understand relationships between texts, while LLMs enable machines to generate and reason with language—and together they power modern AI systems.

Aspect	Embeddings	LLMs
Purpose	Convert text into numerical vectors (semantic representation)	Generate, understand, and manipulate language
Output Type	Fixed-length Numerical vectors (e.g., 50, 300, 768 dims)	Natural language text (sentences, paragraphs, code)
Computation Cost	Lightweight, fast, low cost	Computationally expensive, higher latency
Primary Use Cases	Semantic search, clustering, similarity, recommendations	Chatbots, summarization, translation, content generation
Context Understanding	Captures meaning but limited reasoning	Strong contextual understanding and multi-step reasoning
Determinism	Deterministic (same input → same vector)	Non-deterministic (output may vary)
Storage vs Usage	Stored in vector databases for retrieval	Used at runtime to generate responses
Scalability	Highly scalable for large datasets	Limited by compute and infrastructure
Accuracy vs Creativity	Precise similarity matching	Creative but may hallucinate
Role in Modern Systems	Used for retrieval of relevant information	Used to generate final responses

Applications of vector search and embeddings in real-world scenarios:

Vector search enables systems to move from keyword matching to meaning-based retrieval, making applications more intelligent and user-friendly.

In Modern AI system, It's used in RAG (Retrieval-Augmented Generation)

Embeddings → retrieve relevant data
LLM → generate final answer

Embeddings and vector search are foundational to modern AI systems, enabling scalable, semantic understanding across text, images, and user behavior—powering everything from search engines to personalized recommendations.

Few Examples below:

Application Area	How Embeddings Are Used	Real-World Example
Semantic Search	Convert queries and documents into vectors to find meaning-based matches (not just keywords)	Searching “budget phone with good camera” returns relevant products even if exact words don’t match
Recommendation Systems	Compare user preferences and item embeddings to suggest similar content/products	Netflix recommending movies based on viewing history
Chatbots & Q&A Systems	Retrieve relevant knowledge chunks using embeddings before generating answers	Customer support bots answering FAQs accurately
Document Search & Retrieval	Find similar documents or passages in large corpora	Legal or research databases retrieving relevant case studies
E-commerce Search	Improve product discovery using semantic similarity	Amazon showing similar items or “customers also viewed”
Content Clustering	Group similar articles, news, or posts together	News apps grouping similar stories
Sentiment & Text Analysis	Represent text numerically for downstream ML tasks	Analyzing customer feedback at scale
Image & Multimedia Search	Map images/text into embedding space for cross-modal search	Searching images using text queries (“red sports car”)
Fraud Detection	Identify unusual patterns by comparing behavior embeddings	Banking systems flagging suspicious transactions